



Laporan Praktikum Algoritma & Pemrograman

Semester Genap 2025/2026

SAYA MENYATAKAN BAHWA LAPORAN PRAKTIKUM INI SAYA BUAT DENGAN USAHA SENDIRI TANPA MENGGUNAKAN BANTUAN ORANG LAIN. SEMUA MATERI YANG SAYA AMBIL DARI SUMBER LAIN SUDAH SAYA CANTUMKAN SUMBERNYA DAN TELAH SAYA TULIS ULANG DENGAN BAHASA SAYA SENDIRI.

SAYA SANGGUP MENERIMA SANKSI JIKA MELAKUKAN KEGIATAN PLAGIASI, TERMASUK SANKSI TIDAK LULUS MATA KULIAH INI.

NIM	71251172
Nama Lengkap	Ang, Joshua Alexander Hartono
Minggu ke / Materi	13 / Tipe Data Tuples

PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS KRISTEN DUTA WACANA
YOGYAKARTA
2026

BAGIAN 1: MATERI MINGGU INI (40%)

Tuple Immutable

Tuple pada Python mirip seperti list karena sama-sama digunakan untuk menyimpan banyak data dalam satu variabel dan juga pada setiap datanya memiliki indeks angka. Yang menjadi pembeda antara keduanya adalah tuple bersifat immutable, yaitu isi data dalam tuple tidak bisa diubah setelah dibuat. Karena itu, tuple lebih aman digunakan untuk data yang tetap atau tidak perlu dimodifikasi. Tuple juga digunakan sebagai key dalam dictionary karena termasuk objek hashable, yaitu objek yang nilai hashnya tetap selama program berjalan.

Penulisan tuple dapat dilakukan dengan beberapa cara, seperti menggunakan tanda kurung biasa maupun tanpa tanda kurung selama datanya dipisahkan dengan koma. Contohnya ('a','b','c') atau 'a','b','c'. Jika tuple hanya memiliki satu elemen, maka wajib ditambahkan koma dibelakang elemen tersebut agar dapat dianggap sebagai tuple bukan string biasa.

```
>>> t = ('a','b','c','d','e')
```

Gambar 13.1: Penulisan tuple pada banyak elemen. *Sumber:* <https://shorturl.at/zGVUp>

```
>>> t1 = ('a',)  
>>> type(t1)  
<type 'tuple'>
```

Gambar 13.2: Penulisan tuple pada satu elemen. *Sumber:* <https://shorturl.at/zGVUp>

Python juga menyediakan fungsi bawaan berupa tuple() untuk membuat tuple secara otomatis. Selain itu, tuple juga mendukung banyak operator seperti indexing dan slicing untuk mengambil data tertentu. Karena tuple bersifat immutable, elemen didalamnya tidak bisa diganti langsung. Jika ingin mengganti isi tertentu, maka harus membuat tuple baru dengan menggabungkan bagian lama dan elemen baru.

Membandingkan Tuple

Operator perbandingan pada tuple digunakan untuk membandingkan isi dari dua tuple atau struktur data berurutan lainnya secara berurutan dari elemen pertama. Jika elemen pertama sama, maka Python akan melanjutkan perbandingan ke elemen berikutnya sampai menemukan perbedaan. Proses ini membuat tuple sangat berguna untuk pengurutan data yang memiliki beberapa kriteria.

Konsep ini juga digunakan dalam proses sorting atau pengurutan data di Python. Saat menggunakan fungsi `sort()`, Python akan mengurutkan berdasarkan elemen pertama terlebih dahulu, lalu elemen kedua jika diperlukan, dan seterusnya. Teknik ini dikenal dengan DSU (Decorate, Sort, Underdecorate). Pertama data dihias atau dipasangkan dengan key tertentu, kemudian diurutkan, lalu hasil akhirnya diambil kembali sesuai kebutuhan.

```
1
2 kalimat = 'but soft what light in yonder window breaks'
3 dafkata = kalimat.split()
4 t = list()
5 for kata in dafkata:
6     t.append((len(kata), kata))
7
8 t.sort(reverse=True)
9
10 urutan = list()
11 for length, kata in t:
12     urutan.append(kata)
13
14 print(urutan)
15
16
```

Gambar 13.3: Contoh penggunaan DSU. Sumber: <https://shorturl.at/zGVUp>

Penugasan Tuple

Penugasan tuple pada Python adalah fitur yang memungkinkan beberapa variabel diisi sekaligus dalam satu baris kode. Dengan cara ini, data dari list, tuple, atau urutan lain bisa langsung dibagi ke beberapa variabel sesuai urutannya. Misalnya, jika sebuah list memiliki dua elemen, maka elemen pertama bisa masuk ke variabel pertama dan elemen kedua ke variabel berikutnya.

```

>>> m = [ 'have', 'fun' ]
>>> x, y = m
>>> x
'have'
>>> y
'fun'
>>>

```

Gambar 13.4: Contoh penugasan tuple. *Sumber:* <https://shorturl.at/zGVUp>

Python melakukan proses ini dengan mencocokkan jumlah variabel di sebelah kiri dengan jumlah di sebelah kanan. Jika jumlahnya sama, maka setiap nilai akan otomatis dimasukkan ke variabel sesuai posisinya. Namun, jika jumlah data lebih sedikit atau lebih banyak maka akan muncul error karena Python tidak bisa melakukan pembagian dengan benar.

Keunggulan lain dari tuple assignment adalah dapat digunakan untuk menukar nilai dua variabel tanpa variabel tambahan. Contohnya seperti `a,b = b,a` yang secara otomatis akan menukar isi dari kedua variabel. Selain itu, tuple assignment juga sangat berguna dalam pengolahan data, seperti penggunaan fungsi `split()`. Hasil pemisahan dapat langsung dimasukkan ke dua variabel berbeda dalam satu langkah.

```

>>> email = 'didanendya@ti.ukdw.ac.id'
>>> username, domain = email.split('@')

```

Gambar 13.5: Contoh tuple assignment untuk pengolahan data. *Sumber:*

<https://shorturl.at/zGVUp>

Dictionary dan Tuple

Dictionary pada Python memiliki method bernama `items()` yang digunakan untuk mengambil seluruh isi dari dictionary dalam bentuk key dan value. Hasil dari method ini berupa kumpulan tuple, dimana setiap tuple berisi satu key dan satu value. Karena hasilnya masih berupa objek khusus, biasanya data tersebut diubah terlebih dahulu menjadi list agar lebih mudah diolah.

```

>>> d = {'a':10, 'b':1, 'c':22}
>>> t = list(d.items())
>>> print(t)
[('b', 1), ('a', 10), ('c', 22)]

```

Gambar 13.6: Contoh konversi dictionary dari list tuple. Sumber: <https://shorturl.at/zGVUp>

Multipenugasan dengan dictionaries

Multipenugasan pada dictionary adalah teknik yang menggabungkan penggunaan method `items()`, tuple assignment dan perulangan `for` untuk mengambil key dan value sekaligus dalam satu proses. Dengan cara ini, programmer tidak perlu mengambil key lalu mencari value secara terpisah. Setiap pasangan key dan value dari dictionary akan langsung dimasukkan ke dua variabel berbeda.

```

>>> d = {'a':10, 'b':1, 'c':22}
>>> l = list()
>>> for key, val in d.items() :
...     l.append( (val, key) ) ...
>>> l
[(10, 'a'), (22, 'c'), (1, 'b')]
>>> l.sort(reverse=True)
>>> l
[(22, 'c'), (10, 'a'), (1, 'b')]
>>>

```

Gambar 13.7: Contoh multipenugasan dengan dictionaries. Sumber: <https://shorturl.at/zGVUp>

Kata yang sering muncul

Program ini digunakan untuk mencari kata yang paling sering muncul dalam file teks Romeo dan Juliet Act 2, Scene 2 dengan memanfaatkan dictionary dan list of tuples. Langkah awal adalah membaca file baris per baris, lalu membersihkan tanda baca menggunakan `string.punctuation` agar kata seperti “romeo,” dan “romeo” dianggap sama. Setelah itu, semua huruf akan diubah menjadi huruf kecil menggunakan `lower()`.

Dictionary berfungsi untuk menyimpan pasangan antara kata sebagai key dan jumlah kemunculannya sebagai value. Jika sebuah kata muncul pertama kali, maka nilainya akan dibuat

1. Jika kata tersebut muncul lagi maka nilainya akan ditambah satu. Setelah seluruh kata dihitung, data dictionary akan diubah menjadi list of tuples dalam format (key, value). Tujuannya agar data bisa diurutkan berdasarkan jumlah kemunculan data. Karena Python mengurutkan tuple berdasarkan elemen pertama, maka kata dengan frekuensi tertinggi akan berada di posisi awal setelah proses sorting. Terakhir program akan menampilkan 10 kata teratas yang paling sering muncul.

```
1
2 import string
3 fhand = open('romeo-full.txt')
4 counts = dict()
5 for line in fhand:
6     line = line.translate(str.maketrans('', '', string.punctuation))
7     line = line.lower()
8     words = line.split()
9     for word in words:
10         if word not in counts:
11             counts[word] = 1
12         else:
13             counts[word] += 1
14
15 # urutkan dictionary by value
16 lst = list()
17 for key, val in list(counts.items()):
18     lst.append((val, key))
19
20 lst.sort(reverse=True)
21
22 for key, val in lst[:10]:
23     print(key, val)
24
```

Gambar 13.8 dan 13.9: Program untuk mencari kata yang sering muncul. *Sumber:*

<https://shorturl.at/zGVUp>

Tuple sebagai kunci dictionaries

Tuple dapat digunakan sebagai key dalam dictionary karena bersifat hashable, yaitu nilainya tetap dan bisa dikenali secara unik oleh Python. Karena itu, tuple sangat cocok untuk digunakan sebagai composite key, yaitu key gabungan yang terdiri dari lebih satu nilai. Contohnya pada direktori nomor telepon, dimana nama belakang dan nama depan digabung menjadi satu key

untuk menyimpan nomor telepon seseorang. Dengan cara ini, dua orang yang memiliki nama depan sama dapat dibedakan melalui kombinasi nama lengkapnya. Saat melakukan perulangan pada dictionary dengan key tuple, Python dapat langsung memisahkan isi tuple ke beberapa variabel menggunakan tuple assignment.

```
>>> last = 'nendya'
>>> first = 'dida'
>>> number = '088112266'
>>> directory = dict()
>>> directory[last, first] = number
>>> for last, first in directory:
...     print(first, last, directory[last,first])
...
dida nendya 088112266
>>>
```

Gambar 13.10: Contoh penggunaan tuple sebagai kunci dictionary. *Sumber:*

<https://shorturl.at/zGVUp>

BAGIAN 2: LATIHAN MANDIRI (60%)

Link github : <https://github.com/angjoshua-sudo/Prak-Alpro-2026.git>

SOAL 1

Source code:

```
1  tA = eval(input('tA= '))
2  pembanding = tA[0]
3  for i in range(len(tA)):
4      if pembanding == tA[i]:
5          temp = True
6      else:
7          temp = False
8          break
9
10 print()
11 print(temp)
```

Penjelasan:

Pada bagian awal program, pada baris pertama program meminta input kepada user berupa sebuah tuple dan disimpan ke dalam variabel **tA**. Input menggunakan fungsi awal **eval()** agar data yang dimasukkan dapat dibaca sebagai tuple oleh Python. Setelah itu, pada baris kedua program akan mengambil nilai pertama dari tuple dan menyimpannya ke variabel **pembanding** sebagai acuan untuk pengecekan semua anggota tuple.

Selanjutnya, pada baris ketiga, program menggunakan perulangan **for** dengan **range(len(tA))** untuk memeriksa setiap elemen yang ada dalam tuple satu per satu berdasarkan index. Pada setiap perulangan, program akan membandingkan nilai dari **pembanding** dengan anggota tuple pada posisi tertentu yaitu **tA[i]**. Jika nilainya sama, maka variabel **temp** akan diisi dengan **True**. Namun, jika ditemukan anggota tuple yang nilainya berbeda dengan nilai **pembanding**, maka program akan langsung mengubah nilai **temp** menjadi **False**. Setelah itu, perintah **break** akan menghentikan perulangan. Terakhir, program akan menggunakan **print()** untuk memberikan jarak output agar terlihat lebih rapi, yang kemudian dilanjutkan dengan mencetak isi dari variabel **temp**.

Output:

```
PS C:\Us& C:\Users\Acer\AppData\Local\Programs\Python\Python313\python.exe c:/Users/Acer/OneDrive/Dokumen/belajar-github/minggu13/soal1.py
tA= (90,90,90,90)

True
PS C:\Users\Acer\OneDrive\Dokumen\belajar-github\minggu13> & C:\Users\Acer\AppData\Local\Programs\Python\Python313\python.exe c:/Users/Acer/OneDrive/Dokumen/belajar-github/minggu13/soal1.py
tA= (90,90,90,1000)

False
```

SOAL 2

Source code:

```
1  data = eval(input("Data: "))
2  data_nim = ()
3  data_nama_depan =()
4
5  nama, nim, alamat = data
6  for i in nim:
7      data_nim += (i,)
8
9  bagi_nama = nama.split()
10 nama_depan = bagi_nama[0]
11 for i in range (1, len(nama_depan)):
12     data_nama_depan += (nama_depan[i],)
13
14 print()
15 print(f"NIM      : {nim}")
16 print(f"NAMA      : {nama}")
17 print(f"ALAMAT   : {alamat}")
18 print()
19 print(f"NIM: {data_nim}")
20 print()
21 print(f"NAMA DEPAN: {data_nama_depan}")
22 print()
23 print(f"NAMA TERBALIK: {bagi_nama[::-1]}")
```

Penjelasan:

Pada bagian awal program, baris pertama meminta input dari user berupa tuple berisi nama, NIM, dan alamat, lalu disimpan ke variabel **data**. Baris kedua dan ketiga menyiapkan tuple kosong **data_nim** dan **data_nama_depan** untuk menyimpan hasil proses data. Setelah itu, pada baris kelima program memisahkan isi tuple data menjadi tiga variabel, yaitu **nama**, **nim**, dan **alamat**. Hal ini dilakukan agar setiap bagian data dapat diolah secara terpisah.

Selanjutnya, program menggunakan perulangan **for** pada baris keenam untuk mengambil setiap karakter pada NIM dan memasukkannya satu persatu ke tuple **data_nim**. Pada bagian nama,

program memanfaatkan method **split()** untuk memisahkan nama lengkap menjadi beberapa kata, lalu mengambil nama depan. Setelah itu, program melakukan perulangan mulai dari index 1 agar huruf pertama tidak ikut dimasukkan, kemudian setiap huruf akan dimasukkan ke tuple **data_nama_depan**. Dengan cara ini, data NIM dan nama depan dapat diubah menjadi dormat tuple yang diminta. Pada bagian akhir, program akan menampilkan data asli seperti NIM, nama, alamat menggunakan **print()**. Setelah itu, program akan mencetak tuple NIM, tuple nama depan, serta nama lengkap dengan urutan terbalik menggunakan slicing **[::-1]**. Output juga dibuat terpisah dengan beberapa **print()** kosong agar tampilan lebih rapi.

Output:

```
PS C:\Users\Acer\OneDrive\Dokumen\belajar-github\minggu13> C:\Users\Acer\AppData\Local\Programs\Python\Python313\python.exe c:/Users/Acer/OneDrive/Dokumen/belajar-github
/minggu13/soal2.py
Data: ('Matahari Bhakti Nendya', '22064091', 'Bantul,DI Yogyakarta')

NIM      : 22064091
NAMA     : Matahari Bhakti Nendya
ALAMAT   : Bantul,DI Yogyakarta

NIM: ('2', '2', '0', '6', '4', '0', '9', '1')

NAMA DEPAN: ('a', 't', 'a', 'h', 'a', 'r', 'i')

NAMA TERBALIK: ['Nendya', 'Bhakti', 'Matahari']
```

SOAL 3

Source code:

```
1  file = input('Enter a file name: ')
2  handle = open(file)
3
4  tabel = {}
5
6  for line in handle:
7      if line.startswith('From '):
8          kata = line.split()
9          waktu = kata[5]
10         waktu = waktu.split(":")
11         jam = waktu[0]
12         tabel[jam] = tabel.get(jam,0) + 1
13
14  urut = sorted(tabel.items())
15
16  for i in urut:
17      pukul,jumlah = i
18      print(f"{pukul} {jumlah}")
```

Penjelasan:

Pada bagian awal program, baris pertama meminta user memasukkan nama file yang ingin dibuka. Lalu nama file tersebut akan disimpan ke variabel file. Setelah itu, pada baris kedua, program akan membuka file menggunakan fungsi `open()` dan menyimpannya ke variabel handle agar isi file dapat dibaca baris per baris. Baris keempat membuat dictionary kosong bernama tabel yang nantinya digunakan untuk menyimpan jumlah email berdasarkan jam pengiriman.

Selanjutnya, program menggunakan perulangan `for` untuk membaca setiap baris dalam file. Pada baris ketujuh, program hanya memproses baris yang diawali dengan kata "From " karena baris tersebut berisi informasi email masuk yang diperlukan. Baris kemudian dipecah menggunakan `split()` sehingga setiap kata dapat diakses berdasarkan index. Program mengambil data waktu pada index ke-5, lalu memisahkan format jam menggunakan `split(":")` sehingga hanya bagian jam saja yang diambil.

Pada proses berikutnya, jam yang sudah diperoleh akan digunakan sebagai key dalam dictionary tabel. Dengan method `get()`, program akan menambahkan jumlah kemunculan jam tersebut sebanyak 1 setiap kali ditemukan. Jika jam belum ada di dictionary, maka otomatis dimulai dari 0 terlebih dahulu. Nilai dari setiap key akan terus bertambah sesuai data yang ditemukan.

Pada bagian akhir, isi dari dictionary akan diubah menjadi tuple menggunakan `items()` lalu diurutkan dengan `sorted()` berdasarkan jam. Program kemudian melakukan perulangan kembali untuk menampilkan setiap jam beserta jumlah email yang diterima pada jam tersebut. Variabel `pukul` menyimpan jam, sedangkan `jumlah` menyimpan total email. Setiap data kemudian dicetak dalam format jam dan jumlah.

Output:

```
PS C:\Users\Acer\OneDrive\Dokumen\belajar-github\minggu13> & C:\Users\Acer\AppData\Local\Programs\Python\Python313\python.exe c:/Users/Acer/OneDrive/Dokumen/belajar-github/minggu13/soal3.py
Enter a file name: mbox-short.txt
04 3
06 1
07 1
09 2
10 3
11 6
14 1
15 2
16 4
17 2
18 1
19 1
```